

FINAL COREMARK PIPELINE AXI IMPLEMENTATION

流水线 AXI CoreMark 代码理解与波形验收手册

把 CPU 五段、冒险、AXI 五通道和板级外设落实到具体文件、代码条件与现场波形。目标是能够独立沿时钟解释，而不是背答案。

唯一学习对象：miniRV_pipeline_axi_ego1/

本手册由仓库内 Markdown 生成。代码位置以当前分支为准；现场回答时应直接打开对应 RTL，并以 valid + 时钟沿确认指令是否有效。

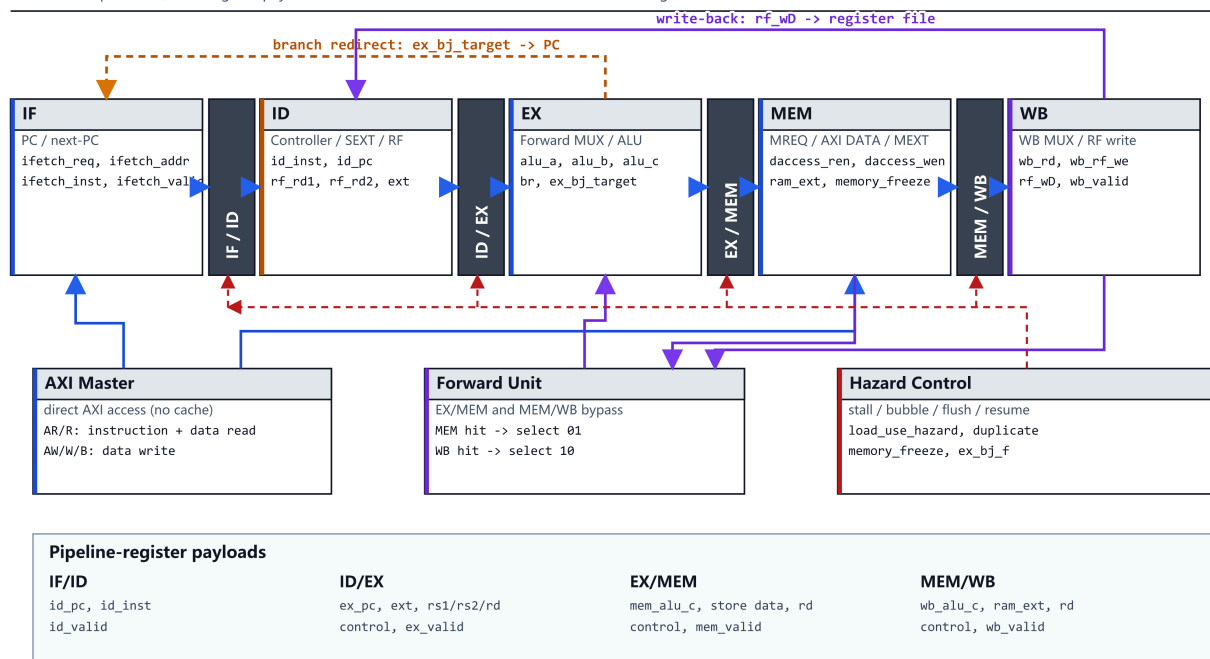
唯一学习对象： `miniRV_pipeline_axi_ego1/`

目标不是背答案，而是能完成：**功能说明 → 找到文件 → 指出具体代码 → 找到波形信号 → 沿时钟解释 → 给出结果证据。**

1. 先建立完整层级

miniRV five-stage pipelined CPU datapath

Block-level report view; exact register payloads and control conditions are listed below the drawing.



Blue: data/AXI | Brown: redirect/control | Purple: forwarding/write-back | Red dashed: hazard control

图 1 最终流水线 CPU 数据通路。阅读时按 IF → ID → EX → MEM → WB 追踪；橙色回路是前递和控制回送。

```
miniRV_SoC
├─ clk_wiz_0          生成 50 MHz 系统时钟
├─ reset_sync        S6/PLL lock 后同步释放复位
├─ cpu_top
│   ├── cpu_core      五级 RV32IM 流水线
│   │   ├── pipeline_regs  IF/ID、ID/EX、EX/MEM、MEM/WB
│   │   ├── Controller    指令译码
│   │   ├── RF            32 个通用寄存器
│   │   ├── SEXT          I/S/B/U/J 立即数
│   │   ├── forward_unit  MEM/WB 前递选择
│   │   ├── ALU
│   │   │   ├── ALU_multicycle  实板组合 ALU + 多周期 M 扩展
│   │   │   └─ ALU_trace      Trace 专用组合实现
│   │   ├── MREQ         load/store byte lane 与 WSTRB
│   │   └─ MEXT          load 对齐和符号/零扩展
│   └─ axi_master        CPU 简单请求转 AXI 五通道
├─ axi_board_soc       AXI Slave + MMIO 地址译码
├─ board_bram          50 KiB IROM + 100 KiB DRAM
├─ simple_uart         115200 8N1 UART
├─ sevenseg_display    八位数码管动态扫描
└─ ila_probe           PC、AXI、复位、UART RX 实板观测
```

记忆主线：

一条指令

- `cpu_core` 的 IF/ID/EX/MEM/WB
- `cpu_top` 请求防重/过期取指过滤
- `axi_master` 五通道握手
- `axi_board_soc` 的 BRAM 或 MMIO
- 响应返回后继续流水或写回

2. 为什么需要这些层，而不是把所有代码写在一起

层	解决的问题	老师问时打开
<code>miniRV_SoC</code>	Trace/实板后端选择、PLL、复位、ILA	<code>src/rtl/miniRV_SoC.v</code>
<code>cpu_top</code>	core 简单接口和 AXI 的时序差异	<code>src/rtl/cpu_top.v</code>
<code>cpu_core</code>	指令怎样经过五级、怎样处理冒险	<code>src/rtl/cpu_core.v</code>
<code>pipeline_regs</code>	每个上升沿哪些数据进入下一段	<code>src/rtl/pipeline/pipeline_regs.v</code>
<code>axi_master</code>	CPU 请求怎样变成五通道握手	<code>src/rtl/axi_master.v</code>
<code>axi_board_soc</code>	地址怎样落到 BRAM/UART/LED/timer	<code>src/rtl/axi_board_soc.v</code>

如果把 AXI 状态机直接放进 core:

- Basic Trace 无法复用核心;
- CPU 必须理解五个 AXI 通道;
- 冒险逻辑和总线握手会混在一起;
- 换成课程 `bram_axi` 或板级 Slave 会更困难。

现在的边界让 `cpu_core` 只理解:

```
取指: req + addr → valid + inst
读数: ren + addr → rvalid + rdata
写数: wen + addr + wdata → wresp
```

3. 一条普通 ADDI 怎样经过五段

示例：

```
addi x2, x1, 3
```

3.1 IF：取指

文件： `cpu_core.v`

关键代码：

```
assign ifetch_req = resume_ifetch | !pause_ifetch;  
assign ifetch_addr = ex_bj_f ? ex_bj_target : pc;
```

含义：

1. 没有 hazard/AXI/M 扩展等待时， `ifetch_req=1`；
2. 正常地址是 `pc`；
3. taken branch 当拍优先请求 `ex_bj_target`；
4. `cpu_top` 和 `axi_master` 完成 AXI AR/R；
5. `ifetch_valid=1` 表示指令真正返回。

PC 只在两种情况下更新：

```
if (ex_bj_f)  
    pc <= ex_bj_target;  
else if (ifetch_valid && !stall_if)  
    pc <= pc + 4;
```

所以“地址线上出现一个值”不代表已经取指，必须同时看返回 `ifetch_valid`。

3.2 IF/ID 上升沿

文件： `pipeline/pipeline_regs.v`

```
if (!stall_if) begin  
    id_pc_r    <= if_pc;  
    id_inst_r  <= if_inst;  
    id_valid_r <= if_valid_in;  
end
```

在上升沿满足：

```
ifetch_valid=1
stall_if=0
flush=0
```

指令才从 IF 进入 ID。波形中上升沿后看到：

```
id_pc    = 该指令 PC
id_inst  = 00308113
id_valid = 1
```

3.3 ID：译码、读寄存器、扩展立即数

文件：

- `Controller.v`
- `RF.v`
- `SEXT.v`

`Controller` 识别：

```
opcode=0010011
funct3=000
→ ADDI
→ alu_op=ALU_ADD
→ alua_sel=RS1
→ alub_sel=EXT
→ sext_op=EXT_I
→ rf_we=1
→ rf_wsel=WB_ALU
```

`RF` 用 `id_rs1=1` 读 x1，`SEXT` 把 `imm=3` 扩展为 32 位。

同拍 WB 若也在写 x1，`cpu_core` 的 `wb_fwd_rs1` 直接选 `rf_wd`，避免寄存器堆 同地址读写行为不确定。

3.4 ID/EX 上升沿

同一个上升沿必须一起保存：

- `ex_pc`
- `ex_rf_rd1/ex_rf_rd2`
- `ex_ext`
- `ex_rs1/ex_rs2/ex_rd`

- `ex_alu_op`
- `ex_alua_sel/ex_alub_sel`
- `ex_rf_we/ex_rf_wsel`
- `ex_valid`

数据和控制必须属于同一条指令。只保存数据、不保存控制会发生“上一条指令的控制操作 下一条指令的数据”。

3.5 EX: 前递、选择操作数、ALU

`forward_unit` 输出:

```
00: 使用 ID/EX 保存值
01: 从 MEM 前递
10: 从 WB 前递
```

然后:

```
alu_a = ex_alua_sel ? ex_pc : fwd_a;
alu_b = ex_alub_sel ? ex_ext : fwd_b;
```

ADDI 得到:

```
alu_a = x1 最新值
alu_b = 3
alu_c = x1 + 3
```

3.6 EX/MEM、MEM、MEM/WB

ADDI 不访问内存, 但仍经过 MEM:

```
ex_alu_c → mem_alu_c → wb_alu_c
```

`mem_valid/wb_valid` 与它一起流动。

3.7 WB: 提交

```
case (wb_rf_wsel)
  WB_ALU: rf_wD = wb_alu_c;
  WB_RAM: rf_wD = wb_ram_ext;
  WB_PC4: rf_wD = wb_pc + 4;
  WB_EXT: rf_wD = wb_ext;
endcase
```

寄存器堆写使能为：

```
wb_rf_we && wb_valid
```

所以 bubble 即使保留 `wb_rf_we=1` 的旧值，也不会产生写回。

4. 五段在代码中的确切位置

段	主要文件	输入	核心工作	输出/边界
IF	<code>cpu_core.v</code> 、 <code>cpu_top.v</code> 、 <code>axi_master.v</code>	<code>pc</code>	产生取指请求、等待 AXI 返回	<code>if_pc/if_inst/if_valid</code>
ID	<code>Controller.v</code> 、 <code>RF.v</code> 、 <code>SEXT.v</code>	<code>id_inst</code>	译码、读寄存器、扩展立即数	ID/EX 全部数据和控制
EX	<code>forward_unit.v</code> 、 <code>ALU*.v</code>	<code>ex_*</code>	前递、ALU、分支目标和比较	EX/MEM
MEM	<code>MREQ.v</code> 、 <code>MEXT.v</code>	<code>mem_*</code>	AXI 数据请求、对齐、扩展	MEM/WB
WB	<code>cpu_core.v</code> 、 <code>RF.v</code>	<code>wb_*</code>	四选一并按序写 rd	架构状态提交

现场判断某条指令在哪一级，只看：

```
id_valid + id_pc
ex_valid + ex_pc
mem_valid + mem_pc
wb_valid + wb_pc
```

不要用 ALU 结果猜级，因为 invalid bubble 也可能留下旧数据。

5. 数据冒险怎样实现

5.1 MEM/WB 前递

文件: `pipeline/forward_unit.v`

匹配条件:

```
写回使能为 1  
rd != x0  
rd == 当前 EX 的 rs1 或 rs2
```

MEM 比 WB 更新, 所以优先 MEM。

MEM 中若是 load, 不能选择 MEM 前递, 因为 `mem_alu_c` 是地址, 不是 load 数据:

```
mem_is_load = mem_ram_rop != RAM_EXT_N;  
fwd_a_ex = mem_rf_we && !mem_is_load && mem_rd == ex_rs1;
```

load 要等数据进入 WB 后用选择码 `10` 前递。

5.2 Load-use

流水线 Load-Use 测试：访存等待、气泡与恢复

程序序列为 LW x2,0(x1) → ADDI x3,x2,1; 本次波形由 memory_freeze 处理等待, load_use_hazard 保持低电平

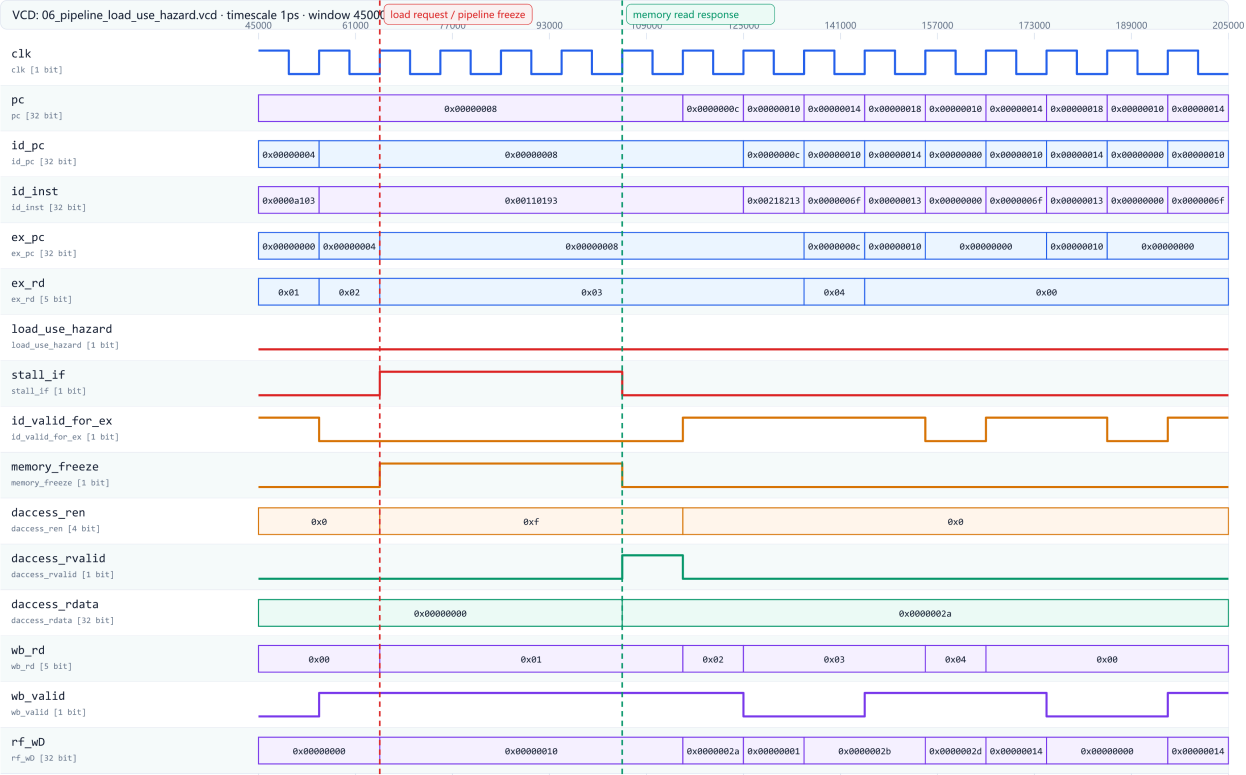


图 2 load-use 定向波形。现场必须结合 valid、目的/源寄存器、stall 与返回数据解释，不能只看 PC 残值。

示例：

```
lw x2, 0(x1)
addi x3, x2, 1
```

命中逻辑：

```
EX 是有效 load
EX rd 非 0
ID 真实使用 rs1/rs2
EX rd == ID rs
```

处理方法：

```
stall_if=1      保持 PC 和 IF/ID
id_valid_for_ex=0 向 ID/EX 注入 bubble
load 继续进入 MEM
AXI 返回后 load 进入 WB
addi 恢复, 并从 WB 前递 x2
```

为什么不是只前递:

```
load 地址在 EX 末尾才算出
数据还必须完成 AXI AR/R
下一条 addi 同拍 EX 时数据不存在
```

5.3 Store 数据前递

```
addi x5,x0,0x55  
sw   x5,0(x1)
```

store 的 rs2 不是 ALU 地址操作数，但仍是要写入内存的数据，所以使用：

```
fwd_store_data
```

并把它锁存为 `mem_rf_rd2`，再交给 `MREQ`。

5.4 同拍 ID/WB 旁路

当一条指令在 WB 写 x1，另一条恰好在 ID 读 x1：

```
wb_fwd_rs1=1 → rf_rd1_fwd=rf_wD
```

这与 EX 前递不同，它发生在 ID，解决寄存器堆同地址读写时序问题。

6. 控制冒险怎样实现

分支/JAL/JALR 到 EX 才确定。

目标：

```
JALR: {alu_c[31:1],1'b0}  
branch/JAL: ex_pc + ex_ext
```

taken:

```
(branch && br) || JAL || JALR
```

实板模式：

```
ex_bj_f = ex_bj_taken  
flush   = ex_bj_f
```

`pipeline_regs` 收到 flush 后：

- IF/ID valid 清零；
- ID/EX valid 清零；
- MEM/WB 不清，因为它们比当前分支更老，必须完成。

同时：

```
pc = ex_bj_target  
ifetch_addr = ex_bj_target
```

旧路径 AXI 取指若后来返回，由 `cpu_top` 的 `ic_pending_word_addr` 比较丢弃。

7. 多周期 M 扩展怎样实现

ALU.v 在实板模式选择 `ALU_multicycle`。

7.1 普通操作

ADD/SUB/AND/OR/shift/compare 是组合逻辑，单次 EX 完成。

7.2 乘除法

启动条件：

```
当前 op 是 M 指令  
operation_issued=0  
所有 worker busy=0
```

启动当拍：

```
operation_start=1  
busy=1  
锁存 op、符号、原始 a/b  
启动 multiplier/divider
```

迭代期间：

```
ex_mul_div_busy=1  
effective_freeze=1  
stall_if/id/ex=1
```

完成后 worker busy 降为 0，结果在 EX/MEM 锁存，`operation_issued` 清除，下一条 M 指令可启动。

必须包含“启动当拍 busy”：

```
busy = operation_start | unit_busy;
```

否则启动 worker 的同一个上升沿，EX/MEM 会先锁存旧结果。

8. AXI 数据访问怎样冻结流水线

访存进入 EX 后：

```
ex_is_ld_st=1  
ld_st_suspend=1
```

完成条件：

```
load: daccess_rvalid=1  
store: daccess_wresp=1
```

等待时：

```
memory_freeze=1  
stall_if=1  
stall_id=1  
stall_ex=1  
stall_mem=1
```

核心保持同一条访存指令和地址/写数据，直到响应。`cpu_top` 在响应拍屏蔽原请求，防止 Master 回到 IDLE 后把同一个请求接受第二次。

9. AXI Master 五通道

AXI4 single-transaction master state machine (no cache)

RTL: axi_master.v | request priority: data write > data read > instruction read

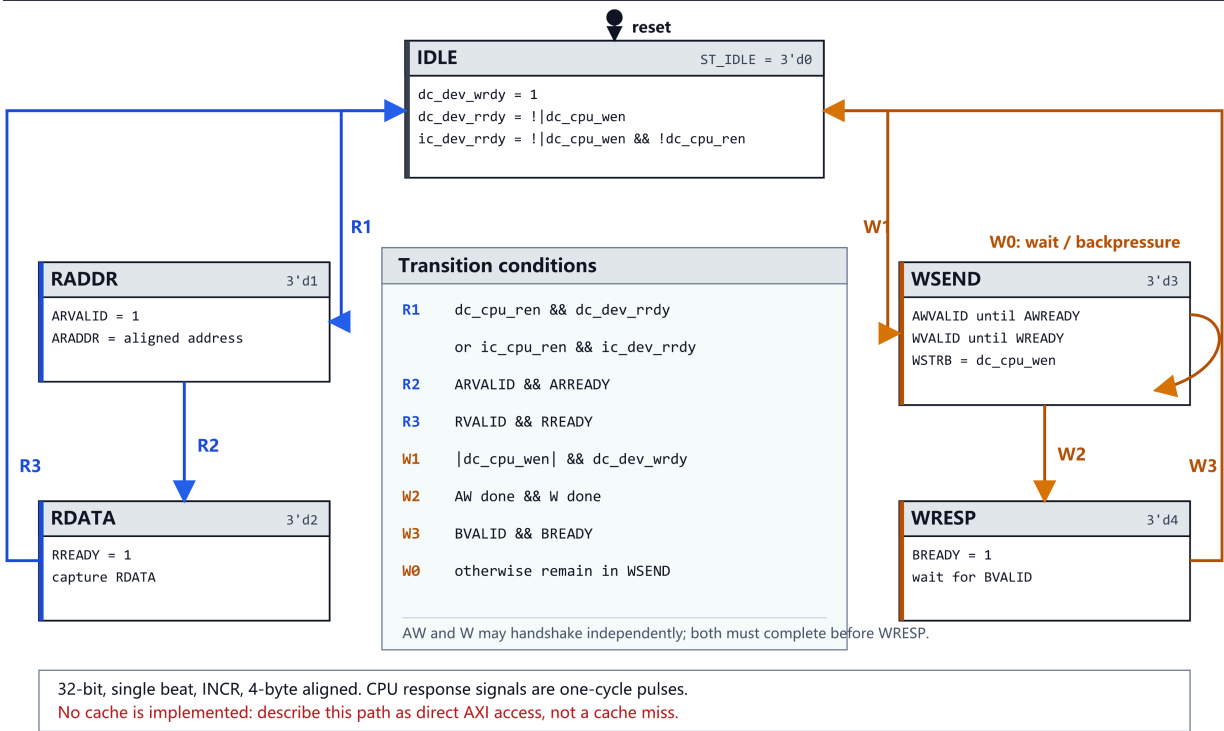


图 3 AXI Master 状态机。读事务由 AR/R 完成，写事务由 AW/W/B 完成。

文件: axi_master.v

状态:

```
ST_IDLE
├ data write → ST_WSEND → ST_WRESP → ST_IDLE
├ data read  → ST_RADDR → ST_RDATA → ST_IDLE
└ inst read  → ST_RADDR → ST_RDATA → ST_IDLE
```

9.1 读事务



图 4 AXI 读通道。只有 VALID 与 READY 同时为 1 的上升沿才完成一次传输。

1. CPU 请求在 `ST_IDLE` 被接受;
2. 锁存并 4 字节对齐地址;
3. `ARVALID=1` 保持到 `ARREADY=1` ;
4. 地址握手后 `RREADY=1` ;
5. `RVALID && RREADY` 时接收 `RDATA` ;
6. 按 `read_is_data` 返回取指口或数据口;
7. CPU 侧 `rvalid` 只拉高一拍。

9.2 写事务

无 Cache 直接 AXI 写事务：AW/W/B 通道握手

写地址和写数据通道可独立握手；两者完成后进入写响应阶段，并向 CPU 返回单周期响应

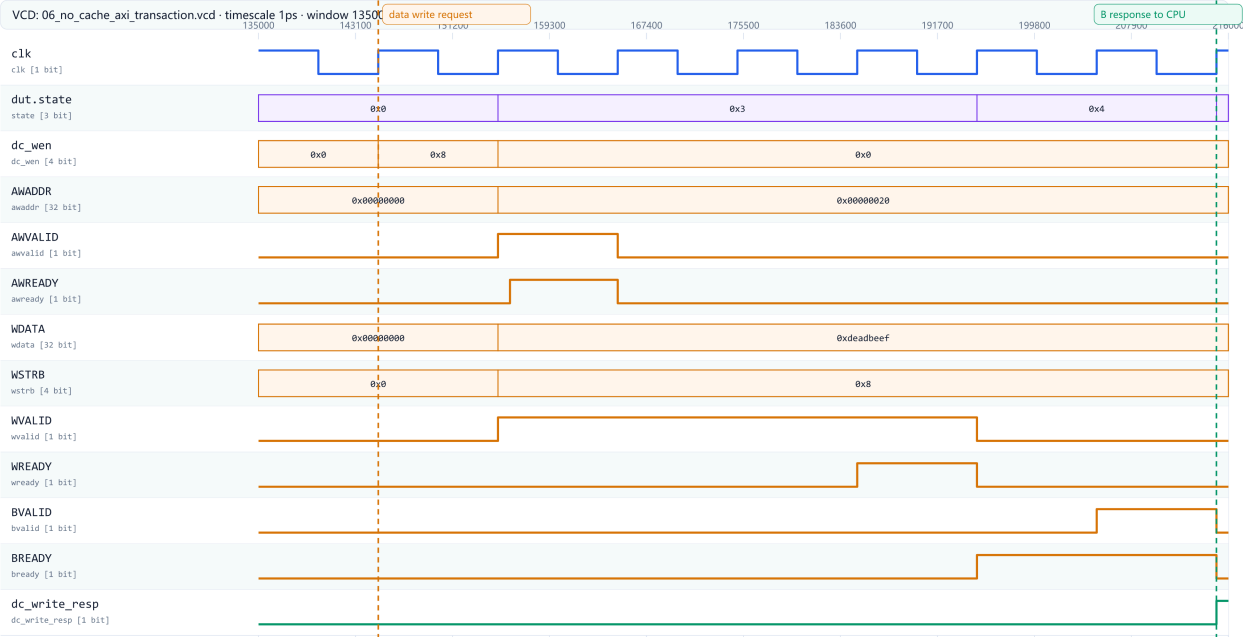


图 5 AXI 写通道。AW 与 W 独立握手，二者完成后再等待 B 响应。

1. CPU `wen!=0` ;
2. 同时发出 AWVALID 和 WVALID;
3. AW 和 W 可以不同拍握手，各自握手后只清自己的 VALID;
4. 两者都完成后进入 WRESP;
5. BVALID && BREADY 后向 CPU 产生 `wresp` 。

AXI 规则：

VALID 由发送方产生
READY 由接收方产生
只有 VALID && READY 的上升沿才算传输
等待期间 VALID、地址、数据必须保持稳定

10. 外设和存储器怎样实现

文件: `axi_board_soc.v`

地址	读	写	代码
<code>0x00000000...</code>	IROM/DRAM	DRAM byte write	<code>board_bram</code>
<code>0xFFFF0000</code>	16-bit switch	-	<code>PERI_ADDR_SWITCH</code>
<code>0xFFFF1000</code>	-	16-bit LED	<code>led_reg</code>
<code>0xFFFF2000</code>	-	32-bit digled	<code>digled_reg</code>
<code>0xFFFF3000</code>	UART RX data	-	<code>uart_rx_data/pop</code>
<code>0xFFFF3004</code>	-	UART TX data	<code>uart_tx_data/start</code>
<code>0xFFFF3008</code>	UART status	-	<code>uart_status</code>
<code>0xFFFF300C</code>	-	UART clear	<code>tx_clear/rx_clear</code>
<code>0xFFFF4000</code>	timer low	-	<code>timer[31:0]</code>
<code>0xFFFF4008</code>	timer high	-	<code>timer[63:32]</code>

10.1 BRAM

`board_bram` :

- IROM 12,800 words;
- DRAM 25,600 words;
- 总计 38,400 words = 150 KiB;
- AXI 字节地址 `[17:2]` 变成 BRAM 字地址;
- BMG 同步读延迟 1 拍, 所以 `memory_read_pending` 记录未返回读。

10.2 LED 和数码管

LED 写:

```
AWADDR=FFFF1000  
WVALID/AWVALID/READY 握手  
根据 WSTRB 更新 led_reg 对应字节  
led = led_reg
```

数码管:

```
AWADDR=FFFF2000 → digled_reg  
sevensseg_display 用 scan_counter[15:13] 轮流选择 8 位  
每位取 value 的 4-bit, 译成段码
```

10.3 UART

发送:

```
软件轮询 UART+8 bit3  
写 UART+4  
axi_board_soc 产生 tx_start 单拍  
simple_uart 锁存 {stop,data,start}  
每 CLKS_PER_BIT 推进一位
```

接收:

```
rx 两级同步  
下降沿 → RX_START  
半 bit 后确认仍为 0  
每个 bit 中心采样 8 位  
停止位为 1 → rx_data、rx_valid  
软件读 UART+0 → rx_pop 清 valid
```

11. CoreMark 软件在做什么

不修改 CoreMark 官方算法源码，避免改变基准口径。需要理解以下入口：

文件	作用
software/c_test/4_coremark/Makefile	选择编译器、源文件和链接脚本
src/common/init_asm.S	设置栈、清 BSS、跳到 C 入口
src/common/ram.lds	定义程序/数据/栈地址
src/common/sc_print.c/.h	UART 输出和基础格式化
src/coremark/core_portme.c/.h	timer、迭代次数、平台适配
src/coremark/src/core_main.c	CoreMark 主控、CRC 和最终报告
core_list_join.c	链表 workload
core_matrix.c	矩阵 workload
core_state.c	状态机 workload
core_util.c	CRC/工具函数
main.mem	编译后的 38,400-word 板级存储镜像

完整正确性不是只看“有输出”，而是：

```
四组 CRC 与参考值一致
Correct operation validated
Iterations 700
FINISH
```

当前 Cache 版实板结果为：

```
Total ticks 717005179
Total time 14 s
CoreMark 48.814
CoreMark/MHz 0.976
```

无 Cache 基线是 32 秒、21.250、0.425/MHz。主频、迭代次数和四组 CRC 相同，所以 Cache 版约 2.30 倍的差异可以直接比较。板卡 `C0DE600D` 照片和 WNS 仍属于旧基线；Cache 版原始截图、时序报告和板卡照片尚待归档。

12. 最终版每个 RTL 文件做什么

文件	必须会说的一句话
<code>defines.vh</code>	全工程控制码、写回选择、访存宽度和 MMIO 地址
<code>miniRV_SoC.v</code>	Trace/实板双后端、PLL/复位、ILA 和顶层连线
<code>cpu_top.v</code>	core/AXI 适配、响应拍防重、过期取指过滤
<code>cpu_core.v</code>	五级流水、冒险、冻结、分支、MEM、WB
<code>pipeline/pipeline_regs.v</code>	四组级间寄存器及 stall/flush/valid
<code>Controller.v</code>	opcode/funct 到全部控制信号
<code>RF.v</code>	ID 双读、WB 单写、x0 恒 0
<code>SEXT.v</code>	I/S/B/U/J 立即数拼接
<code>pipeline/forward_unit.v</code>	MEM/WB RAW 前递，禁止 load 的 MEM 地址误前递
<code>ALU.v</code>	Trace/FPGA ALU 选择器
<code>pipeline/ALU_trace.v</code>	Trace 组合 RV32IM 运算
<code>pipeline/ALU_multicycle.v</code>	实板组合 ALU、多周期 worker 和 freeze busy
<code>multiplier.v</code>	32 拍移位加法乘法
<code>divider.v</code>	32 拍恢复除法
<code>MREQ.v</code>	store WSTRB/写数据对齐和 load 读请求
<code>MEXT.v</code>	load 返回数据的 byte offset 和符号扩展
<code>axi_master.v</code>	三路 CPU 请求到 AXI 五通道
<code>axi_board_soc.v</code>	BRAM/MMIO AXI Slave
<code>board_bram.v</code>	IROM/DRAM 两个 BMG 的连续地址适配
<code>simple_uart.v</code>	115200 8N1 TX/RX
<code>sevensseg_display.v</code>	8 位十六进制动态扫描

13. 原始波形一键生成

必须在有 `iverilog` 和 `vvp` 的 Ubuntu/Linux 服务器执行：

```
cd ~/miniRV_pipeline_axi_ego1
sudo apt install -y iverilog # 已安装可跳过
bash tests/generate_report_vcd.sh
```

输出：

```
docs/course-report/vcd/06_pipeline_load_use_hazard.vcd
docs/course-report/vcd/07_pipeline_five_stage_forward_branch.vcd
docs/course-report/vcd/06_no_cache_axi_transaction.vcd
docs/course-report/vcd/09_board_peripheral_mmio_uart.vcd
```

终端必须出现：

```
PASS: pipeline_hazard_tb
PASS: pipeline_flow_tb
PASS: axi_master_tb
PASS: board_peripheral_tb
```

上述四个 testbench 已在 Ubuntu GitHub Actions/Icarus 回归 `30446791756` 中实际执行并全部通过。仓库内四份 VCD 就是该次构建产物：

VCD	字节数	SHA-256
<code>06_pipeline_load_use_hazard.vcd</code>	41,172	<code>81a4ea95274f7cefbfb6249941472417b3cd4f4e2ec9f016f0a01f6682ef3965</code>
<code>07_pipeline_five_stage_forward_branch.vcd</code>	38,357	<code>604f751f34215df13d82bc4b2bde61a64eb4d0d35c4f89702b384bef2f25df8</code>
<code>06_no_cache_axi_transaction.vcd</code>	4,979	<code>773e05a8c059d3c7b8bfec58bc9f13bb12e978e703a051fc8ec39f7f9262957e</code>
<code>09_board_peripheral_mmio_uart.vcd</code>	633,295	<code>bd517134842eeba48e14f41ab553f40d17575d4dec1217288e808f68a40697fc</code>

本分支新增 Cache 后，本地 Icarus 回归另外生成了三份逐层证据。它们证明 Cache testbench、AXI Master burst 和板端 BRAM burst 的仿真行为。当前 Cache RTL 另已在 Linux 课程 AXI Trace 中通过 45/45，见 [miniRV_pipeline_cache_axi_report.md](#)；但 VCD/Trace 都不能被表述为 Vivado 综合或实际下板结果：

VCD	字节数	SHA-256
10_cache_refill_hit_uncached.vcd	6,749	5e3e07b8df56c4f563d984509079e7f8ab81b0bc8a493d8cd6dc1480ac34576c
08_axi_cacheline_burst.vcd	7,376	59281b2a249999858c0fa22592fedbf50891cf4d072dd002e669c9089adb4713
11_board_bram_burst.vcd	8,446	f8dd0ec617ac29cfa46bb4d994862b8aa1fe02be3f5954bbec41458f6e50db21

14. 在 Surfer/GTKWave 中现场找信号

14.1 基本操作

1. File → Open, 选择 `.vcd` ;
2. 左侧 Scope 展开 testbench, 再展开 `dut` ;
3. 搜索信号名并双击/Add to Wave;
4. PC、地址、指令和数据设为 Hex;
5. valid/ready/stall/flush 设为 Binary;
6. 先加 `clk`, 把光标放在上升沿;
7. 每解释一拍, 都按“沿前条件 → 沿后寄存器变化”说。

不要把绿色粗条当成高电平。总线频繁变化、缩放太远时也会显示成粗条, 必须放大并看 Value。

14.2 五级流动与分支波形

打开:

```
07_pipeline_five_stage_forward_branch.vcd
```

Scope:

```
pipeline_flow_tb / dut
```

加入:

```
clk rst
ifetch_addr ifetch_valid ifetch_inst
id_pc id_inst id_valid
ex_pc ex_valid ex_rs1 ex_rs2 ex_rd
mem_pc mem_valid mem_rd
wb_pc wb_valid wb_rd
forward_a_sel forward_b_sel
alu_a alu_b alu_c
ex_bj_taken ex_bj_target ex_bj_f flush
stall_if stall_id stall_ex stall_mem
```

重点找:

1. PC 0/4/8 的三条算术指令同时占据不同级;
2. `forward_a_sel/forward_b_sel` 从 00 变 01/10;

3. PC=0x0c 的 BEQ 在 EX 时 `ex_bj_f=1` ;
4. 下一上升沿 `id_valid/ex_valid` 对错误路径变 0;
5. PC=0x10 的 x5=99 不得到 WB 提交。

14.3 Load-use 波形

打开:

```
06_pipeline_load_use_hazard.vcd
```

加入:

```
id_pc id_valid ex_pc ex_valid mem_pc mem_valid wb_pc wb_valid
load_use_hazard
stall_if stall_id stall_ex stall_mem
id_valid_for_ex
ld_st_suspend ld_st_done memory_freeze
daccess_ren daccess_addr daccess_rvalid daccess_rdata
forward_a_sel rf_wD
```

重点找:

1. `lw` 在 EX、相关 `addi` 在 ID;
2. `load_use_hazard=1` ;
3. IF/ID 保持而 ID/EX valid=0;
4. AXI 模型等待期间 `memory_freeze=1` ;
5. `daccess_rvalid=1` 后 load 进入 WB;
6. `addi` 恢复且 `forward_a_sel=10` 。

14.4 AXI 波形

无 Cache 历史基线打开:

```
06_no_cache_axi_transaction.vcd
```

当前 Cache 版本必须另外打开:

```
08_axi_cacheline_burst.vcd
```


不能只展示 `06_no_cache_axi_transaction.vcd` 就声称已经解释当前 Cache。 `06` 适合解释单拍 AXI 握手； `08` 才包含 `ARLEN=3`、四拍 `RDATA`、 `read_beat/read_buffer` 和 `RLAST`。

Scope:

```
axi_master_tb / dut
```

加入:

```
clk rst state read_is_data
ic_cpu_rdy ic_dev_rdy ic_dev_rvalid
dc_cpu_rdy dc_dev_rdy dc_dev_rvalid
dc_cpu_wen dc_dev_wrdy dc_dev_wresp
m_axi_araddr m_axi_arvalid m_axi_arready
m_axi_arlen m_axi_arsize m_axi_arburst
m_axi_rdata m_axi_rvalid m_axi_rready m_axi_rlast
m_axi_awaddr m_axi_awvalid m_axi_awready
m_axi_wdata m_axi_wstrb m_axi_wvalid m_axi_wready
m_axi_bvalid m_axi_bready
read_len read_beat read_buffer
```

重点找:

- ARVALID 等待 ARREADY 时地址保持;
- Cache refill 时 `ARLEN=03` , Uncached 外设读时 `ARLEN=00` ;
- 四次 `RVALID && RREADY` 分别写入 `read_buffer` 的四个 32-bit lane;
- 第四拍 `RLAST=1` 后才产生 `ic_dev_rvalid/dc_dev_rvalid` ;
- RVALID/RREADY 同拍才返回 CPU;
- AW 已握手后 AWVALID 清零, 但 WVALID 在 backpressure 下保持;
- WDATA/WSTRB 在 WREADY=0 时不变;
- B 握手后才产生 `dc_dev_wresp` 。

14.5 外设波形

打开:

```
09_board_peripheral_mmio_uart.vcd
```

Scope:

```
board_peripheral_tb / dut
```

加入：

```
s_axi_awaddr s_axi_awvalid s_axi_awready  
s_axi_wdata s_axi_wstrb s_axi_wvalid s_axi_wready  
s_axi_bvalid s_axi_bready  
s_axi_araddr s_axi_arvalid s_axi_arready  
s_axi_rdata s_axi_rvalid s_axi_rready  
write_accept write_is_memory read_is_memory  
led_reg digled_reg timer  
uart_tx_start uart_tx_data uart_tx_busy  
uart_rx_pop uart_rx_data uart_rx_valid uart_status  
U_uart/tx_active U_uart/tx_bit_index U_uart/tx_shift  
U_uart/rx_state U_uart/rx_clock_count U_uart/rx_bit_index U_uart/rx_shift
```

按地址找：

```
FFFF1000 → LED=00A5  
FFFF2000 → digled=600D600D  
FFFF0000 → switch=1234  
FFFF4000 → timer low  
FFFF3004 → UART TX 55  
FFFF3008 → UART status  
FFFF3000 → UART RX 41 并 pop
```

15. 在 Vivado 仿真器中现场找波形

如果老师不接受预先截图，可直接在 Vivado 打开 testbench：

1. Flow Navigator → Simulation → Run Behavioral Simulation;
2. Simulation Settings 把顶层设为： - `pipeline_flow_tb` - `pipeline_hazard_tb` - `axi_master_tb` - `board_peripheral_tb`
3. Scopes 中展开 `dut` ;
4. Objects 中搜索上面清单的信号;
5. 右键 Add to Wave Window;
6. Restart;
7. Run All;
8. 右键数据总线 → Radix → Hexadecimal;
9. Zoom Fit 后再放大到目标上升沿。

如果这些 testbench 没加入 Vivado Simulation Sources:

```
Add Sources
→ Add or create simulation sources
→ 选择 miniRV_pipeline_axi_ego1/tests/*.v
```

只加入当前要看的 testbench，避免多个 top 同时运行。

16. 实板 ILA 能看什么

当前 `ila_probe[199:0]` 位于 `miniRV_SoC.v`，适合板级定位：

位	信号
199:192	ARLEN, Cache refill 为 03
191	RLAST
190:187	WSTRB
186	原始 <code>rx</code>
185	UART 同步后 rx
184:183	UART RX state
182	UART RX valid
181:174	UART RX data
173	PLL lock
172	sys_rst
171:140	PC
139	ifetch_req
138	ifetch_valid
137:106	ARADDR
105:74	RDATA
73:70	AR/R valid-ready
69:38	AWADDR
37:6	WDATA
5:0	AW/W/B valid-ready

ILA 看不到当前完整的 ID/EX/MEM/WB 内部信号，所以：

- 五级、前递、load-use、flush：用 Behavioral Simulation/VCD；
- 板上是否复位、PC 是否前进、AXI 是否握手、UART RX 是否收到：用 ILA。

不要在老师面前声称 ILA 截图中存在没有探出的 `id_pc/ex_pc`。

17. 波形回答固定模板

每次回答只按五句话：

1. **哪条指令**：用 stage PC + valid 确认；
2. **在哪一级**：ID/EX/MEM/WB；
3. **为什么变化**：译码、相关、分支或 AXI 握手条件；
4. **这个上升沿做什么**：保持、推进、bubble 或 flush；
5. **怎样证明正确**：WB、store response、目标 PC、UART/CRC/测试 PASS。

示例：

当前 PC=4 的 lw 在 EX，PC=8 的 addi 在 ID，ex_rd=2 与 id_rs1=2 命中，所以 load_use_hazard 拉高。这个上升沿 IF/ID 保持，ID/EX valid 清零形成 bubble，lw 继续进入 MEM 并等待 AXI。R 响应后 lw 进入 WB，addi 恢复到 EX，forward_a_sel=10 选择 WB 的 42，最终写 x3=43。

18. 下一次验收前必须自己完成的练习

不看讲义，分别对着源代码回答：

1. 从 `miniRV_SoC` 找到 UART TX 引脚最终由哪个 `always` block 驱动；
2. 从 `cpu_core` 找到 `lw -> addi` 的暂停条件和 bubble 注入位置；
3. 从 `pipeline_regs` 说明 flush 为什么只影响 IF/ID 与 ID/EX；
4. 从 `forward_unit` 说明为什么 MEM load 不能前递；
5. 从 `axi_master` 指出 AW/W 不同拍握手的代码；
6. 从 `axi_board_soc` 指出 `0xFFFF3008` 的返回值；
7. 在 VCD 中找到同一逻辑的信号变化；
8. 用五句话模板完整解释，不使用“这里大概是”“应该是”。

19. 流水线 A/B 组验收分工

完整逐段讲稿见 `PIPELINE_AB_GROUP_SCRIPT.md`。现场分工不是把工程割裂成两个互不相干的模块，而是在同一条请求链上交接：

```
A 组
cpu_core: IF → ID → EX → MEM → WB
      前递 / load-use / stall / flush / M busy
          |
          | ifetch_* / daccess_*
          ▼

B 组
cpu_top: ICache / DCache、MMIO Uncached
      ↓
axi_master: D write > D read > I read、4-beat line refill
      ↓ AXI AR/R/AW/W/B
axi_board_soc: 150 KiB BRAM + switch/LED/数码管/UART/timer
```

19.1 A 组：理想流水线划分

A 组按以下顺序讲：

1. IF: PC 和 `ifetch_req/addr`;
2. IF/ID: 锁存指令、PC 和 `valid`;
3. ID: Controller、RF、SEXT;
4. ID/EX: 锁存操作数、rs/rd 和全部后级控制;
5. EX: 前递、ALU、分支判断;
6. EX/MEM: 锁存 ALU 结果和已经前递后的 store 数据;
7. MEM: MREQ/MEXT 和 `daccess_*`;
8. MEM/WB、WB: 四选一结果写回 RF。

A 组必须用 `07_pipeline_five_stage_forward_branch.vcd` 展示多条指令同时处于不同级，并用 `id/ex/mem/wb_pc + valid` 确认真实在途指令。

19.2 A 组：冒险处理

冲突	处理	关键代码	波形
ALU RAW	MEM/WB 前递, MEM 优先	<code>forward_unit.v</code> 、 <code>fwd_a/fwd_b</code>	<code>forward_*_sel</code>
load-use	保持 PC/IF-ID, ID/EX 注入 bubble	<code>load_use_hazard</code> 、 <code>id_valid_for_ex</code>	<code>06_pipeline_load_use_hazard.vcd</code>
store 数据相关	rs2 单独走 <code>fwd_store_data</code>	<code>cpu_core.v</code> 、 <code>MREQ.v</code>	<code>daccess_wdata/wen</code>
taken branch	EX 改 PC, flush IF/ID 与 ID/EX	<code>ex_bj_f</code> 、 <code>flush</code>	<code>ex_bj_target</code> 、各级 valid
AXI 长延迟	<code>memory_freeze</code> 保持在途状态	<code>ld_st_suspend/done</code>	<code>stall_*</code> 、 <code>daccess_rvalid/wresp</code>
MUL/DIV 长延迟	busy 冻结, issued 防重复启动	<code>ALU_multicycle.v</code>	<code>ex_mul_div_busy</code>

A 组不能把 load-use 说成“全部级停一拍”。正确过程是：相关指令暂时不能进入 EX，老 load 继续进入 MEM；AXI 真正未完成时才触发更广的 freeze。

19.3 B 组：ICache/DCache 的准确边界

当前版本已经打开 `ENABLE_ICACHE/DCACHE`，并实例化 `ICache.v/DCache.v`。两者均为 64-line direct-mapped、16-byte line，包含 tag/data/valid、hit/miss 和 refill 状态机。DCache 使用 write-through/no-write-allocate，因此有 write FSM 但不需要 dirty bit 与 write-back FSM。

正确回答：

I/D read hit 直接返回；miss 对齐到 16 byte，用 `ARLEN=3` 四拍 refill。store 始终 write-through，写命中同步更新对应 byte lane；外设地址 Uncached、单拍访问。

完整逐状态讲解见 `CACHE_IMPLEMENTATION_AND_DEFENSE.md`。

19.4 B 组：连接关系

取指：

```
cpu_core.ifetch_*
→ cpu_top.cpu2ic/ic2cpu
→ ICache hit, 或 miss/refill
→ axi_master.ic_cpu_*
→ AXI AR/R (refill 为四拍)
→ BRAM
→ IF/ID
```


数据：

```
cpu_core.daccess_*
→ cpu_top.cpu2dc/dc2cpu
→ DCache hit/read refill/write-through/Uncached
→ axi_master.dc_cpu_*
→ AXI AR/R 或 AW/W/B
→ BRAM 或 MMIO
```

Cache 设备侧 FSM 只在 **REQ** 状态发一次请求，响应后回 IDLE，因此持久的 core 请求不会重复进入 AXI。taken branch 期间旧 ICache refill 可以完成安装，但回到 IDLE 后会用当前 PC 重新检查 hit，不把旧 line 当作新指令。

axi_master 的仲裁优先级是：

```
data write > data read > instruction read
```

axi_board_soc 按地址连接：

地址	对象
低地址 150 KiB	board_bram
0xFFFF0000	switch
0xFFFF1000	LED
0xFFFF2000	数码管
0xFFFF3000-300C	UART
0xFFFF4000/+8	64 位 timer

B 组用 **10_cache_refill_hit_uncached.vcd** 解释 Cache 内部，用 **08_axi_cacheline_burst.vcd** 解释 Master 四拍拼接，用 **11_board_bram_burst.vcd** 解释板端 Slave burst，并用 **09_board_peripheral_mmio_uart.vcd** 解释地址译码和外设寄存器。

19.5 两组交接句

A 组结束：

```
MEM 级已经形成 daccess_ren/wen/addr/wdata；请求如何经过 I/D 侧接口、AXI 和 地址译码到达 BRAM 或外设，由 B 组继续说明。
```

B 组开始：

我从 `ifetch_*` 和 `daccess_*` 接口继续。I/D Cache 先判断 hit; read miss 经过 AXI 四拍 refill, store write-through, MMIO 则绕过 Cache 单拍访问。

两组都必须知道: `daccess_rvalid/wresp` 一方面是 B 组 AXI 事务的完成证据, 另一方面 会解除 A 组流水线中的 `memory_freeze`。

20. B 组：把 Cache、AXI 和外设波形连成一条行为链

这一节不是要求在一张波形中同时看到全部模块。现有四份 VCD 是四个不同层次的 定向 testbench：

VCD	所在层次	能证明什么	证据边界
10_cache_refill_hit_uncached.vcd	ICache/DCache 内部	tag/index/hit、refill、write-through、Uncached	真实 AXI 五通道
08_axi_cacheline_burst.vcd	AXI Master	仲裁、ARLEN、四拍拼接、AW/W/B	板端地址译码
11_board_bram_burst.vcd	板端 AXI Slave + BRAM	连续地址、同步 BRAM 延迟、RLAST	CPU 的 hit/miss
09_board_peripheral_mmio_uart.vcd	板端 AXI Slave + MMIO	LED/数码管/UART/timer 的寄存器行为	Cache 内部数组

现场正确说法：

我按同一事务的四个层次关联证据，而不是声称四份独立 testbench 是一条同时采集的全系统波形。Cache 波形证明为何发请求，Master 波形证明请求怎样变成 AXI，Slave 波形证明地址怎样到 BRAM/外设，最后再用响应信号说明流水线何时继续。

20.1 先按地址判断“这次应该看哪条路径”

CPU行为	地址示例	Cache行为	AXI行为	Slave行为
取指	0x00000024	ICache hit 或四拍 refill	AR/R burst	读 BRAM
普通 load	0x00000044	DCache hit 或四拍 refill	AR/R burst	读 BRAM
普通 store	0x00000044	write-through	AW/W/B 单拍写	写 BRAM
读拨码	0xFFFF0000	DCache Uncached	ARLEN=0	返回 sw
写 LED	0xFFFF1000	不分配 Cache Line	AW/W/B	更新 led_reg
写数码管	0xFFFF2000	不分配 Cache Line	AW/W/B	更新 digled_reg
UART RX	0xFFFF3000	Uncached	ARLEN=0	返回字节并 rx_pop
UART TX	0xFFFF3004	write-through 到外设	AW/W/B	tx_start 单拍
UART状态	0xFFFF3008	Uncached	ARLEN=0	返回 uart_status
读计时器	0xFFFF4000/+8	Uncached	ARLEN=0	返回 timer 低/高 32 位

这里必须主动解释：MMIO 如果进入 Cache，switch、UART状态和timer可能返回旧值， UART RX 的读副作用也可能丢失或重复。因此 0xFFFF_xxxx 绝不能分配到 DCache。

20.2 ICache miss、refill、再命中的逐拍对应

打开 10_cache_refill_hit_uncached.vcd，Scope 选择：

```
cache_tb / U_icache
```

加入：

```
clk rst state
cpu_ren cpu_raddr cpu_index cpu_tag cpu_hit cpu_rvalid cpu_rdata
miss_line_addr miss_index miss_tag
dev_ren dev_rdy dev_raddr dev_rvalid dev_rdata
```

用地址值定位：

```
cpu_raddr = 00000024    第一次访问，miss
dev_raddr = 00000020    16-byte 对齐后的 line 地址
cpu_raddr = 0000002C    同一 line 的另一个 word，hit
```

沿时钟解释：

时刻	Cache状态/信号	具体行为
T0	state=IDLE, cpu_ren=1, cpu_hit=0	valid 或tag不匹配，不能给CPU指令
T1	state=REQ, dev_ren=1, dev_raddr=0x20	保存miss的tag/index，并请求整条16-byte line
T2	dev_ren && dev_rdy	Master接受请求，Cache进入WAIT
T3	state=WAIT, dev_rvalid=1	128-bit line写入data/tag/valid数组
T4	回到 IDLE, cpu_hit=1	cpu_raddr[3:2]=01，选择第二个word并拉高 cpu_rvalid
T5	地址改成 0x2C，cpu_hit=1	选择第四个word；dev_ren 不再出现

这一段最后要指着 ic_request_count=1 说：

第一次miss只产生一次设备请求；同一line内第二次取指直接hit，这就是空间局部性。

20.3 DCache load miss、load hit与流水线freeze

同一文件选择：

```
cache_tb / U_dcache
```

加入：

```
state cpu_addr cpu_ren cpu_index cpu_tag cpu_hit cpu_cacheable
cached_read_hit cpu_rvalid cpu_rdata
req_addr req_uncached miss_index miss_tag
dev_ren dev_rrdy dev_raddr dev_rvalid dev_rdata
```

用 `cpu_addr=0x00000044` 定位。行为是：

时刻	DCache	AXI/CPU含义
T0	<code>cpu_ren=F, cpu_hit=0, cpu_cacheable=1</code>	load miss, CPU尚未收到完成
T1	<code>state=RREQ, dev_raddr=0x40, dev_uncached=0</code>	请求四拍Cache Line
T2	<code>dev_ren && dev_rrdy</code>	请求被Master接受，进入RWAIT
T3	<code>dev_rvalid=1</code>	128-bit line安装到DCache
T4	回IDLE后 <code>cached_read_hit=1</code>	从 <code>addr[3:2]=01</code> 选择 <code>BBBB0001</code>
T4	<code>cpu_rvalid=1</code>	对接到core的 <code>daccess_rvalid</code> ，解除 <code>memory_freeze</code>

Cache testbench中没有完整 `cpu_core`，所以 `memory_freeze` 要在流水线仿真或代码中 关联解释：

```
ld_st_done    = daccess_rvalid || daccess_wresp;
memory_freeze = ld_st_suspend && !ld_st_done;
```

不能说“Cache miss直接暂停AXI”；正确说法是：

miss让 `daccess_rvalid` 暂时不返回，core看到访存事务未完成而冻结在途流水级；refill完成后DCache给出 `rvalid`，同一个完成事件解除freeze。

20.4 DCache write-through和WSTRB怎样在波形中体现

仍看 `10_cache_refill_hit_uncached.vcd`，用以下值定位：

```
cpu_addr = 00000044
cpu_wdata = 0000AA00
cpu_wen = 0010
```

逐项说明：

1. 这是已缓存line上的写命中；
2. `cpu_wen=0010` 表示只写byte lane 1；
3. `merge_bytes` 把原来的 `BBBB0001` 更新为 `BBBBAA01`；
4. 同时进入 `ST_WREQ`，设备侧出现：

```
dev_waddr = 00000044
dev_wdata = 0000AA00
dev_wen = 0010
```

5. `dev_wresp=1` 时 `cpu_wresp=1`，store才算架构完成；
6. 随后再次读取同一地址，`cpu_rdata=BBBBAA01` 且没有新refill。

答辩结论：

写命中既更新Cache副本，又用同一WSTRB写入下级，所以叫write-through；写miss直接写下级、不取整行，所以叫no-write-allocate；因此不需要dirty bit。

20.5 MMIO Uncached怎样从Cache波形过渡到AXI波形

在 `10_cache_refill_hit_uncached.vcd` 中用以下值定位：

```
cpu_addr = FFFF0002
cpu_cacheable = 0
req_addr = FFFF0000
req_uncached = 1
dev_uncached = 1
cpu_rdata = 00001234
```

这说明DCache完成了三件事：

1. 判断地址不允许缓存；
2. 只做32-bit字对齐，不做16-byte line对齐；
3. 返回时只使用 `dev_rdata[31:0]`，不写tag/data/valid数组。

随后切换到 `08_axi_cacheline_burst.vcd`，用 `dc_uncached=1` 定位。必须指出：

```
read_len = 0
m_axi_arlen = 00
```

所以Uncached读只消费一次 `RVALID && RREADY`，而不是四拍refill。

20.6 AXI Master四拍refill逐拍讲法

打开 `08_axi_cacheline_burst.vcd`：

```
axi_master_tb / dut
```

加入：

```
state read_is_data read_len read_beat read_buffer read_buffer_next
ic_cpu_ren ic_cpu_raddr ic_dev_rrdy ic_dev_rvalid ic_dev_rdata
dc_cpu_ren dc_cpu_raddr dc_cpu_uncached dc_dev_rvalid dc_dev_rdata
m_axi_araddr m_axi_arlen m_axi_arsize m_axi_arburst
m_axi_arvalid m_axi_arready
m_axi_rdata m_axi_rvalid m_axi_rready m_axi_rlast
```

先用 `ic_cpu_raddr=0x00000003` 定位取指refill：

观察点	波形现象	对应总线规则
地址对齐	<code>m_axi_araddr=0x00000000</code>	取Cache Line首地址
长度	<code>m_axi_arlen=03</code>	四个beat
每拍大小	<code>m_axi_arsize=010</code>	每beat 4 byte
突发类型	<code>m_axi_arburst=01</code>	INCR，地址每拍+4
地址握手	<code>ARVALID && ARREADY</code>	Slave接受整次burst描述
数据第0拍	<code>RDATA=11110000, read_beat=0</code>	写 <code>buffer[31:0]</code>
数据第1拍	<code>RDATA=22220001, read_beat=1</code>	写 <code>buffer[63:32]</code>
数据第2拍	<code>RDATA=33330002, read_beat=2</code>	写 <code>buffer[95:64]</code>
数据第3拍	<code>RDATA=44440003, RLAST=1</code>	写 <code>buffer[127:96]</code>
完成	<code>ic_dev_rvalid=1</code>	完整128-bit line一次返回ICache

一定要说明为什么代码使用 `read_buffer_next`：

最后一拍RDATA和 RLAST 在同一个沿到达。如果只读取旧 `read_buffer` , 最后32位尚未 写进去;
`read_buffer_next` 把当前RDATA先组合进line, 再在该拍返回完整128位数据。

再用数据读和取指同时出现的片段解释仲裁:

```
dc_cpu_ren=1 且 ic_cpu_ren=1
dc_dev_rrdy=1
ic_dev_rrdy=0
m_axi_araddr=00000010
```

结论:

Master优先级为D写、D读、I读; MEM级数据事务会冻结流水线, 所以先完成数据侧事务。

20.7 板端Slave怎样把四拍请求变成四次BRAM地址

打开 `11_board_bram_burst.vcd` :

```
board_burst_tb / U_dut
```

加入:

```
s_axi_araddr s_axi_arlen s_axi_arvalid s_axi_arready
s_axi_rdata s_axi_rvalid s_axi_rready s_axi_rlast
read_active read_addr_reg read_len_reg read_beat_reg
memory_read_accept memory_read_continue memory_read_issue
memory_read_pending memory_read_data
```

逐拍对应:

时刻	Slave内部	BRAM/AXI含义
T0	<code>ARVALID && ARREADY, ARLEN=3</code>	锁存起始地址和burst长度
T1	<code>memory_read_issue=1</code>	向同步BRAM发第0个字地址
T2	<code>memory_read_pending=1</code>	BRAM数据有效，产生第0拍RVALID
T3	<code>RVALID && RREADY</code> 且未结束	<code>read_addr_reg += 4, read_beat_reg += 1</code>
后续	重复发起BRAM读	地址依次为base、base+4、base+8、base+12
最后一拍	<code>read_beat_reg == read_len_reg</code>	<code>s_axi_rlast=1</code>
最后一拍握手	<code>RVALID && RREADY && RLAST</code>	清除 <code>read_active</code> ，burst结束

Master波形中的四个R beat和Slave波形中的四次BRAM地址是——对应的：

```
Cache Line word0 ← base+0
Cache Line word1 ← base+4
Cache Line word2 ← base+8
Cache Line word3 ← base+12, RLAST
```

20.8 LED和数码管：AXI写如何变成物理输出

打开 `09_board_peripheral_mmio_uart.vcd`，先加入：

```
test_stage
s_axi_awaddr s_axi_awvalid s_axi_awready
s_axi_wdata s_axi_wstrb s_axi_wvalid s_axi_wready
write_accept write_is_memory
s_axi_bvalid s_axi_bready
led_reg led digled_reg dig_en dig_seg
```

定位 `test_stage=1`。

LED事务：

```
AWADDR = FFFF1000
WDATA  = 000000A5
WSTRB  = 0011
```

需要沿时钟说清楚：

1. AW和W各自在 `VALID && READY` 时被接受；
2. `write_accept=1` 且 `write_is_memory=0`，说明不是BRAM写；

3. 地址译码命中LED;
4. `WSTRB=0011` 更新LED低16位;
5. `led_reg` 由 `0000` 变为 `00A5`, 物理端口 `led` 同步变为 `00A5`;
6. Slave拉高 `BVALID`, Master收到B响应后才向DCache/core报告写完成。

数码管事务:

```
AWADDR = FFFF2000
WDATA  = 600D600D
WSTRB  = 1111
```

`digled_reg` 立即保存 `600D600D`, 但 `dig_en/dig_seg` 是扫描输出, 会按位循环变化。所以波形中应该用 `digled_reg=600D600D` 证明软件写入值正确, 用 `dig_en/dig_seg` 证明显示驱动正在扫描, 不能要求段选始终等于一个固定值。

20.9 Switch和timer: AXI读如何返回会变化的外设值

仍看 `09_board_peripheral_mmio_uart.vcd`, 定位 `test_stage=2`, 加入:

```
s_axi_araddr s_axi_arvalid s_axi_arready
s_axi_rdata s_axi_rvalid s_axi_rready s_axi_rlast
read_is_memory sw timer
```

Switch:

```
ARADDR = FFFF0000
read_is_memory = 0
RDATA = 00001234
RLAST = 1
```

Timer:

```
ARADDR = FFFF4000
RDATA = timer[31:0]
```

两次timer读之间等待8拍, 第二次返回值应更大。这个波形正好说明为什么timer必须 Uncached: 如果DCache返回第一次读到的旧副本, 第二次值不会增长。

20.10 UART TX: 一次MMIO写怎样产生串行10个bit

定位 `test_stage=3`, 加入:

```
s_axi_awaddr s_axi_wdata s_axi_wstrb write_accept
uart_tx_start uart_tx_data uart_tx_busy
U_uart/tx_active U_uart/tx_shift U_uart/tx_bit_index U_uart/tx_clock_count tx
```

事务入口：

```
AWADDR = FFFF3004
WDATA = 00000055
WSTRB = 0001
```

行为链：

```
AXI写握手
→ 地址译码命中UART+4
→ uart_tx_data=55, uart_tx_start产生单拍脉冲
→ U_uart锁存为10-bit帧: start + 8 data + stop
→ tx_active/uart_tx_busy保持
→ tx_bit_index逐位增加
→ tx引脚输出8N1、LSB-first串行波形
```

`0x55=01010101`，LSB-first发送，所以数据位在波形中呈交替0/1。老师问“为什么串口只收到U”时，`0x55`的ASCII正是 'U'，这不是乱码。

20.11 UART RX：串行A怎样经过状态寄存器和数据寄存器

定位 `test_stage=4` 和 `test_stage=5`，加入：

```
rx uart_debug_rx_sync uart_debug_rx_state uart_debug_rx_valid uart_debug_rx_data
uart_rx_valid uart_rx_data uart_status uart_rx_pop
s_axi_araddr s_axi_rdata s_axi_rvalid s_axi_rready
```

接收链路：

```
rx出现start bit
→ rx_state从IDLE进入START/DATA/STOP
→ rx_bit_index从0到7
→ 收到41
→ uart_rx_valid=1, uart_status[0]=1
```

软件先读状态：

```
ARADDR = FFFF3008
RDATA[0] = 1
```

再读数据：

```
ARADDR = FFFF3000
RDATA[7:0] = 41
uart_rx_pop = 1
```

读 `0xFFFF3000` 不仅返回数据，还消费当前RX字节，所以必须Uncached。否则Cache hit 可能跳过真实MMIO读，`uart_rx_pop` 不会出现，软件会重复看到旧字符。

20.12 一条store到LED的完整端到端讲法

老师如果要求不分文件、直接讲完整链路，可按以下七句：

1. MEM级产生 `daccess_wen/wdata/addr`，地址为 `0xFFFF1000`；
2. DCache判断它是MMIO，不做Cache Line分配；store统一走write-through写路径；
3. AXI Master在仲裁中选择D写，锁存地址、数据和WSTRB；
4. AW、W通道可以不同拍握手，未握手的VALID和payload必须保持；
5. `axi_board_soc` 发现地址不属于BRAM而命中LED，按WSTRB更新 `led_reg`；
6. Slave给B响应，Master产生 `dc_dev_wresp`，DCache再产生 `daccess_wresp`；
7. `daccess_wresp` 解除 `memory_freeze`，流水线允许后续指令继续。

对应波形证据：

```
daccess_wen
→ dc_dev_wen
→ AWVALID/AWREADY + WVALID/WREADY
→ write_accept
→ led_reg
→ BVALID/BREADY
→ dc_dev_wresp
→ daccess_wresp
```

20.13 一条load从BRAM miss到写回的完整端到端讲法

1. MEM级发出load地址，DCache第一次检查 `cpu_hit=0`；
2. DCache保存tag/index和16-byte对齐地址，进入RREQ/RWAIT；
3. AXI Master发送 `ARLEN=3`，`ARSIZE=2`，`ARBURST=INCR`；
4. Slave对BRAM发出base、base+4、base+8、base+12四次读取；

5. 四个R beat由Master拼成128-bit，最后一拍 `RLAST=1`；
6. DCache安装data/tag/valid，回到IDLE重新检查原请求；
7. 这次hit，按 `addr[3:2]` 选一个32-bit word并产生 `daccess_rvalid`；
8. core进行lb/lh/lw符号或零扩展，load进入WB；
9. 依赖指令通过load-use bubble和WB前递获得正确值。

20.14 波形回答固定句式

每解释一个事务都按这六句，不容易漏：

1. **行为**：CPU正在取指、load、store还是访问哪个外设；
2. **Cache判断**：地址、cacheable、index/tag、hit/miss；
3. **请求握手**：哪个 `VALID && READY` 接受了什么地址/数据；
4. **下级动作**：BRAM读写或哪个MMIO寄存器/脉冲发生变化；
5. **响应握手**：R/B何时完成，`RLAST` 或 `WSTRB` 说明什么；
6. **流水线结果**：`rvalid/wresp` 何时解除freeze，指令如何继续或写回。

严禁只说“波形这里动了，所以成功”。必须把信号变化解释成协议事件：

错误：这里ARVALID变高，然后RDATA出来了。

正确：Master保持ARVALID和地址，直到ARREADY同拍完成地址握手；随后每次RVALID与RREADY同拍才消费一个beat，第四拍RLAST表明整条Cache Line结束，此后Master才向ICache/DCache给出一次完整line响应。

20.15 B组现场练习题

不看答案，对着VCD完成：

1. 在 10 中指出 `0x24` 为什么对齐成 `0x20`，并找出随后 `0x2C` 未再次refill；
2. 在 10 中用 `0010` 和 `BBBBAA01` 证明byte merge；
3. 在 10 和 08 中分别证明MMIO Uncached以及 `ARLEN=0`；
4. 在 08 中指出四个RDATA最终位于128-bit line的哪四段；
5. 在 08 中指出AW已完成、W仍等待时哪些信号必须保持；
6. 在 11 中逐拍读出四个BRAM地址，并指出最后一拍RLAST；
7. 在 09 中从AW/W握手讲到 `led_reg=00A5` 再讲到B响应；
8. 在 09 中解释 `digled_reg` 与扫描输出 `dig_en/dig_seg` 的区别；
9. 在 09 中从 `tx_start` 讲到串行 `0x55`；
10. 在 09 中从串行 `0x41` 讲到状态读、数据读和 `rx_pop`。